

Report for the Course on Cyber-Physical Systems and IoT Security

Emilio Risi (matricola 2122841)

Title of the reference paper

Authentication of IoT Device and IoT Server Using Secure Vaults (M. Gupta and P. Kumaraguru, IEEE TrustCom/BigDataSE 2018)

Objectives

The objective taken from the course's project list is to implement the authentication protocol of the reference paper [1], Secure Vault, on a real constrained device. The extension agreed on top of it is to measure what the protocol costs there against the three baselines the paper compares it with - a classical pre-shared-key AES-128 challenge-response, the single rotating password that Gupta & Kumaraguru's Figure 3 is drawn against, and an asymmetric ECC scheme - and so to test the paper's central efficiency claim: that Secure Vault buys key rotation, and with it resistance to next-password prediction, for a small constant-factor premium over plain symmetric authentication, while asymmetric authentication costs orders of magnitude more. A second objective, smaller but not optional for a security protocol, is to show that the implementation refuses replayed, forged and tampered requests rather than merely accepting genuine ones.

The target is the relative cost between the protocols rather than absolute agreement with the paper's numbers, for the reason given under Experiments. The body ends with the references; Appendices A to C are reference material - terminology and notation, a per-phase cost breakdown with the interface the runs were driven from, and reproduction steps - and are not needed to follow the argument.

System Setup

The testbed is one microcontroller and one Linux host on the same WiFi network. There is no external measurement instrument: the device times its own cryptographic work with its microsecond clock and reports the statistics over HTTP, which is what makes the setup reproducible from a single command - the exact sequence is in Appendix C. Cryptographic acceleration as built: AES in hardware, SHA in hardware, elliptic-curve P-256 arithmetic in software.

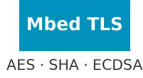
Software and hardware under test

PlatformIO + Arduino core



Builds, uploads and monitors; the Arduino core supplies the WiFi stack and the WebServer behind the device API. The optimisation level is pinned and recorded in every dataset.

Mbed TLS



The single source of every primitive measured [7]: AES-128-ECB [3], HMAC-SHA256 [5] and ECDSA P-256 [4]. One library for all four protocols means the comparison is between protocols, not implementations.

Python 3 host harness (uv, pycryptodome, Matplotlib)



Real participants, not drivers: `test_client.py` mirrors the device's key material and verifies every reply. `benchmark.py` drives both measurement paths and writes the dataset; `analyze.py` charts it.

ESP32-C3 SuperMini (silkscreen HW-466AB)



The device under test [6], shown in Figure 1: single-core 32-bit RISC-V at 160 MHz, 400 KB SRAM, 4 MB flash, WiFi and BLE, on a 22 x 18 mm board with 13 castellated GPIOs and USB-C on the chip's native USB. It accelerates AES and SHA in hardware but has no elliptic-curve accelerator - the single most important fact for reading the results.

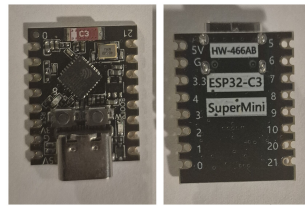


Figure 1. The device under test, both faces: the ESP32-C3 QFN package, crystal, PCB antenna and USB-C; and the silkscreen (HW-466AB, ESP32-C3 SuperMini) with the 13 castellated GPIOs. About 22 x 18 mm.

What was built

Four authentication protocols run on the device, each as a pair of HTTP endpoints. Classical AES-128 and Secure Vault are the paper's two symmetric schemes, the single rotating password is the baseline that Figure 3 of the paper compares against, and ECDSA P-256 is the asymmetric one. The host is a full cryptographic peer for all four: it holds the matching key material, mirrors the state that changes between sessions and verifies every reply, so a cycle counts as successful only when both sides authenticate.

The firmware times each cryptographic phase - one step of a protocol, such as an AES call or the HMAC - against a microsecond clock and accumulates count, min, max, mean and standard deviation on-chip with Welford's algorithm; `GET /api/energy` returns them, and `GET /api/benchmark` runs a chosen number K of repetitions back to back. Those are the two paths Figure 2 draws. Nothing is timed on the host, where WiFi latency would swamp the measurement.

The hardware cost is one board at EUR 4.12 and a USB-C cable; the device times itself, so there is no instrumentation to buy. The paper's Arduino needed 2.5 ms for one AES-128 operation and a board costing about the same today does it in microseconds with a radio attached, so the class of device the paper calls resource-constrained became far less constrained at the same price.

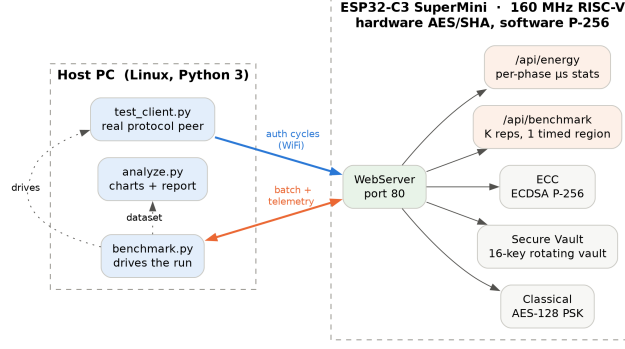


Figure 2. The testbed. Two independent measurement paths reach the device: the orange path runs K repetitions inside a single timed region on-chip, the blue path drives real, cryptographically verified authentication cycles over WiFi. Both report the same quantity - the device's own summed cryptographic phases - so the two are directly comparable.

Experiments

What the four protocols do

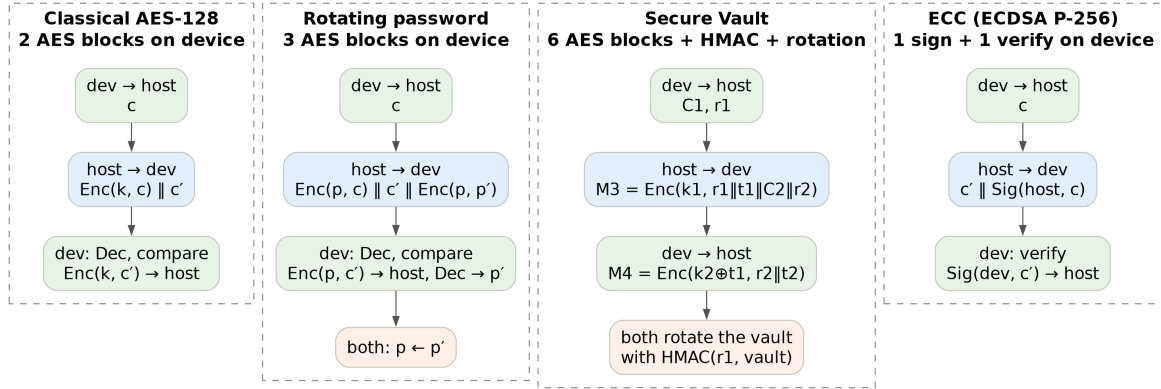


Figure 3. The four authentication exchanges as implemented, with the device's cryptographic work per cycle. The volume of that work differs sharply between them, which is the main reason the measured ratios depart from the paper's. Notation is defined in the table that follows.

All four, drawn side by side in Figure 3, are challenge-response protocols exposed as a pair of HTTP endpoints, an `/init` that issues the device's challenge and a `/verify` that consumes the host's reply. **Classical** shares one AES-128 key k : the host answers c with $\text{Enc}(k, c) \parallel c'$, and the device decrypts to verify the host, then encrypts c' to answer it - the two AES operations the paper charges to its simplest scheme. The **single rotating password** does the same under the current password p and adds a third block, $\text{Enc}(p, p')$, carrying the next password; both sides replace p with p' after a successful session. **Secure Vault** is the paper's three-way exchange [1], with the vault set to sixteen 128-bit keys and four drawn per challenge, the parameters its security analysis is stated for: the device issues $C1$ and $r1$, the host replies $M3 = \text{Enc}(k1, r1 \parallel t1 \parallel C2 \parallel r2)$, four blocks, and the device answers $M4 = \text{Enc}(k2 \oplus t1, r2 \parallel t2)$, two blocks; afterwards both sides rotate every key of the vault using $\text{HMAC}(r1, \text{vault})$, so a captured exchange does not compromise later sessions. The ESP32 device is the HTTP server, so relative to Gupta & Kumaraguru the roles are mirrored: the ESP32 acts as the 'IoT server' (issuing the first challenge, decrypting $M3$, encrypting $M4$), while the Python host acts as the 'IoT device'. Because the protocol's cryptographic cost is symmetric, this role reversal leaves

the measured workload on the ESP32 identical to the workload of the 'IoT device' in the original paper. **ECC** uses ECDSA over P-256: the host returns $c' \parallel \text{Sig}(\text{host}, c)$, and the device verifies it and answers with $\text{Sig}(\text{device}, c')$.

Notation used in the exchanges (repeated in Appendix A)

Symbol	Meaning
c, c'	16-byte random challenges issued by the device and by the host
k	the pre-shared AES-128 key of the Classical scheme
p, p'	current and next password of the rotating scheme, 16-byte AES keys
$\text{Enc}(k, x), \text{Dec}(k, x)$	AES-128 ECB encryption / decryption of x under key k , per 16-byte block
$C1, C2$	vault challenges: four indices into the 16-key vault, one set per side
$k1, k2$	the XOR of the four vault keys that $C1$ and $C2$ select
$r1, r2$	random nonces paired with $C1$ and $C2$
$t1, t2$	random contributions to the session key $t = t1 \text{ XOR } t2$
$M3, M4$	the paper's message numbers ($M1, M2$ are the request and $C1$ with $r1$)
$\parallel, \text{XOR}$	concatenation; bitwise exclusive-or
$\text{HMAC}(r1, \text{vault})$	HMAC-SHA256 over the whole 256-byte vault, keyed with $r1$
$\text{Sig}(X, x)$	ECDSA P-256 signature of x under party X 's private key

How the tests were run

Every result comes from one invocation of `make benchmark` against the flashed board, which writes a self-describing dataset. For each protocol in turn the harness resets the device's statistics, runs a short warm-up that is excluded, then a batch of K repetitions on-chip, then 10 real authentications over WiFi that the host verifies cryptographically. It records the run's provenance, power model, build flags and batch sizes. A run that fails a check is still written, so it can be diagnosed, but the harness exits non-zero and the report generator refuses to present it.

The experiment was built up iteratively. The pipeline was first developed against a mock device that runs all four protocols with the same keys and cryptography but invented timings, so a broken client or analysis could be caught without hardware; its datasets are marked mock and the report generator refuses them. Runs on the board were repeated while diagnosing what the numbers showed (the gap between batch and network figures under Results, for one), with the dashboard the firmware serves (Appendix B, Figure B.2) driving the same endpoints interactively. The dataset reported here is the final run, and every figure is regenerated from it.

The primary measurement is the batch path: the device runs K back-to-back repetitions of a protocol on-chip ($K = 1000$ for the symmetric protocols, 15 for ECC, which is far slower). Each repetition also has to build the messages a server would have sent, and that peer simulation is deliberately left outside the timed phases: what is reported is the sum of the phases the device itself executes, which is the quantity the reference paper counts *at the IoT device side*. The whole-region figure, peer simulation included, is recorded in the dataset as an upper bound and runs 20-40% higher.

Every phase is timed at 1 us granularity, which on the Classical path - two AES-128 blocks, tens of microseconds in total - would carry percent-level error on a single shot; averaging over K samples drives the residual below a microsecond. The loop writes through a volatile sink and each iteration consumes the previous one's output, so the optimiser cannot delete work the benchmark then reports as fast.

Because a batch loop could in principle time something a real authentication never does,

the network cycles are an independent second path, and the harness rejects the run if the two disagree beyond the observed dispersion, or if any protocol produced no successful network cycle at all. A batch iteration runs *warm*, code and tables already in the flash cache; a network cycle runs *cold*, after the HTTP path has evicted them. The cost of an empty instrumented region is measured as the floor below which no phase is trustworthy (0.50 us in this run).

How cost is measured

The device does not measure energy. It measures time, and energy is derived from it - the same choice the reference paper made [1], so the two sets of figures come from the same procedure rather than merely being compared. The object of study is the *algorithm*, not this board's power draw: a meter in the supply line measures the whole system - regulator, radio, idle current, and how efficiently this silicon implements AES and SHA - so comparing protocols through it would largely compare hardware implementations of primitives, and its resolution (1-2 Hz for a USB meter, low kHz for an INA219 current sensor) cannot see operations that finish in microseconds. Direct measurement remains a **complementary experiment**, capturing the radio and idle draw this model excludes.

The starting point is the definition of electrical energy E over a time t in which the power P , supply voltage V times current I , is treated as constant:

$$E = P \cdot t, \quad P = V \cdot I$$

The paper's section VI-A cites Prasithsangaree et al. [2] and computes energy as average current times supply voltage times execution time, on an Arduino drawing 19.9 mA at 5 V. That fixes its power constant. The second is taken from this chip's datasheet [6], whose Table 5-8 puts a WiFi-associated ESP32-C3 running at 160 MHz between 23 mA and 28 mA depending on peripheral clocks; 25 mA is the midpoint, declared rather than measured:

$$P_{\text{paper}} = 5.0 \text{ V} \times 19.9 \text{ mA} = 99.5 \text{ mW} \quad P_{\text{device}} = 3.3 \text{ V} \times 25 \text{ mA} = 82.5 \text{ mW}$$

With E in microjoules (uJ), P in milliwatts (mW) and t in microseconds (us), the divisor carries the unit conversion:

$$E = \frac{P \times t}{1000}$$

This single expression produces every energy number in this report and is reproducible by hand from any latency and power constant printed here. It lives in `power_model.h` on the device and is mirrored in `benchmark.py`; the harness checks that the two agree before it writes a dataset.

Why the ratios are the trustworthy part

Absolute energies depend on a power constant that was declared rather than measured, so the report leads with ratios, in which it cancels: a ratio between two protocols is a pure ratio of measured times, comparable against the paper's Arduino and immune to an error in either constant.

$$\frac{E_A}{E_B} = \frac{P \cdot t_A}{P \cdot t_B} = \frac{t_A}{t_B}$$

Results and Discussion

Did it reproduce?

The ordering reproduces and no quantitative claim does. Symmetric authentication is cheap and asymmetric authentication is more expensive by orders of magnitude, which is the paper’s qualitative argument. Every ratio is larger than the paper’s, and the one the paper leads with - Secure Vault against the rotating password, the only baseline that also rotates key material - comes out reversed: dearer here, where the paper has it cheaper. The rest of this section accounts for that, and the larger part turns out to be a property of the paper’s cost model rather than of this hardware. The table below lists every comparison.

Cost ratios against the reference paper, every comparison it makes; the first is the one Gupta & Kumaraguru’s Figure 3 leads with. Its column is computed from the energies in Gupta & Kumaraguru’s Table 1 and section VI-A.

Comparison	Measured	Paper	Measured / paper	Outcome
Secure Vault / rotating password	3.98x	0.87x	4.6x	direction reversed
Secure Vault / Classical	5.99x	1.30x	4.6x	same direction, larger
ECC / Secure Vault	969.30x	170.00x	5.7x	same direction, larger
ECC / Classical	5,803.45x	221.00x	26.3x	same direction, larger

Measured cost

Measured per-cycle cryptographic cost at the device side over the batch ($K = 1000$, 15 for ECC). Latencies in microseconds, energies in microjoules derived from the mean. Min, mean and standard deviation are exact over every iteration; the median is over the most recent 256, which is the whole batch for ECC.

Protocol	Min	Median	Mean	SD	@ 99.5 mW	@ 82.5 mW
Classical AES-128	56.00	57.00	57.60	8.87	5.73	4.75
Rotating password	84.00	85.00	86.74	12.40	8.63	7.16
Secure Vault	334.00	337.00	344.86	63.11	34.31	28.45
ECC (ECDSA P-256)	332,158	334,307	334,273	1,465.16	33,260	27,578

The distributions are not symmetric. Every median sits within a microsecond or two of its minimum, and the standard deviation comes from a few iterations interrupted by the WiFi stack and the scheduler rather than from measurement noise. Reading the table by median moves no ratio here by more than a few per cent, so the means are reported untrimmed.

Warm batch cross-checked against real network cycles. Both columns are the device’s own summed cryptographic phases.

Protocol	Batch (warm)	Network cycles (cold)	Cycles	Failed	Difference
Classical AES-128	57.60 us	179.40 us	10	0	+211.5%
Rotating password	86.74 us	223.40 us	10	0	+157.6%
Secure Vault	344.86 us	552.80 us	10	0	+60.3%
ECC (ECDSA P-256)	334,273 us	337,950 us	10	0	+1.1%

No batch figure exceeds its network-cycle cost, so the batch loop performs the work a real authentication performs and the reported figure is a protocol cost, not an artefact

of the loop. The difference between them is itself a result: one cold cycle costs 3.1x the warm figure for Classical AES-128 and 1.01x for ECC (ECDSA P-256). The cause is the flash cache: a network cycle runs once, right after the HTTP and WiFi path has evicted the cryptographic code and its tables, where in a batch only the first iteration pays that. Radio interference was ruled out: the gap is unchanged after improving the received signal strength from -89 to -70 dBm. The warm figure is the cost of the cryptography, and every energy figure here uses it. That choice is not decisive: a device that authenticates rarely pays the cold cost every time, and recomputed from the cold column the four comparisons above become 2.47x, 3.08x, 611x, 1,884x - same direction, same reversal, so no verdict depends on which column is read.

The same comparisons across the 3 most recent runs, oldest first; the last is the one reported above. Each is an independent invocation of the harness against the board (run 1 = 20260911T141924Z, run 2 = 20260911T194720Z, run 3 = 20260911T211347Z).

Comparison	Run 1	Run 2	Run 3	Spread
Secure Vault / rotating password	3.84x	3.84x	3.98x	4%
Secure Vault / Classical	5.71x	5.44x	5.99x	10%
ECC / Secure Vault	1,013x	998x	969x	5%
ECC / Classical	5,783x	5,431x	5,803x	7%

Nothing moves by more than a few per cent across 3 independent runs, so these are properties of the implementation and not of one lucky execution.

Why the numbers differ

The quantitative agreement is not close, and the first reason is that the paper’s cost model does not depend on message size. Gupta & Kumaraguru’s Table 1 [1] charges 248.75 uJ for one AES-128 operation however many blocks that operation covers. Counted the paper’s way this implementation matches it exactly: one decryption and one encryption per Secure Vault cycle at the device side. But the protocol it specifies in section IV encrypts $M3 = r1 \parallel t1 \parallel C2 \parallel r2$, which is four AES blocks, and $M4 = r2 \parallel t2$, which is two. Six blocks are executed and two operations are billed.

That matters because the rotating password’s three operations are single blocks, so it is billed at cost while Secure Vault is not. Re-costing both per block, using the paper’s own figures and changing nothing else, Secure Vault comes to 1,641.75 uJ against the rotating password’s 746.25 - a ratio of 2.20x rather than 0.87x. Most of the reversal reported here is therefore already implied by the paper’s own data once message size is accounted for; it is a property of the accounting, not of this platform.

A second term is unbilled outright. Deriving the session key from the four selected vault keys and rotating all sixteen afterwards cost 4.9 us and 20.7 us, together 7% of Secure Vault’s cycle. Their Table 1 has no row for either: it charges the vault update as one HMAC operation and says nothing about the XOR work on either side of it, so that work is invisible in the paper’s cost model.

The third reason is the platform, and it accounts for the remainder. AES and SHA run on accelerators here while P-256 does not, so every comparison that crosses the symmetric/asymmetric line is inflated, and the HMAC-to-AES relation the paper’s whole argument rests on inverts - quantified below. Neither reason is a defect in the implementation: each protocol performs exactly the operations the paper attributes to it, checked call by call against the firmware.

Cryptographic operations per cycle at the device side, counted in the firmware against the reference paper’s accounting.

Protocol	Implemented here	Paper	Agrees
Classical AES-128	2 AES ops (2 blocks)	2 AES ops	yes
Single rotating password	3 AES ops (3 blocks)	3 AES ops	yes
Secure Vault	2 AES ops (6 blocks), 1 HMAC	2 AES ops, 1 HMAC	operations yes, blocks no
ECDSA P-256	1 sign, 1 verify	"ECC public key encryption"	substitution, disclosed

Absolute energies are far below the paper’s and are not expected to match: Classical AES-128 is 87x cheaper here and ECC (ECDSA P-256) 3.3x. The divergence is largest where the accelerators bite hardest and smallest for ECC, which runs in software on both platforms - the clearest evidence that the accelerators, not the clock alone, drive the gap, and the reason the comparison is made on ratios.

The paper’s own baseline, measured

Figure 3 of Gupta & Kumaraguru [1] compares Secure Vault not with the plain two-AES scheme but with the **single rotating password**, costed at 746.25 uJ for three AES-128 operations. That is the like-for-like comparison, because it too rotates key material after every session, and on the paper’s numbers Secure Vault wins it - $646.75 / 746.25 = 0.87x$ - by swapping the third AES for an HMAC.

So this baseline was implemented and measured too - three AES-128 block operations on the device, the password replaced after every successful session. **It does not reproduce.** Measured here Secure Vault costs 3.98x the rotating password, against the paper’s 0.87x: the ordering the paper’s central figure asserts is reversed on this platform.

The reason is visible in the phase breakdown of Appendix B, Figure B.1, and it is not a fault in either protocol. The paper’s argument rests on one HMAC being cheaper than one AES operation - on its Arduino, 149.25 uJ against 248.75 uJ, a factor of 0.60 - so replacing an AES with an HMAC saves energy. On this chip that relation inverts: a single AES-128 block costs 28.9 us because it runs on a hardware accelerator, while the HMAC-SHA256 over the vault costs 126.8 us, or 4.4 times an AES block. The swap that saved energy on an unaccelerated processor costs energy on an accelerated one.

The six-block accounting above compounds it, as does the HMAC’s input, the whole 256-byte vault. The paper’s efficiency claim is therefore conditional on symmetric primitives being unaccelerated and comparable in cost, and that condition fails on a modern microcontroller with an AES engine - the class of device the protocol targets.

How faithfully the protocol is implemented

The Secure Vault exchange described under Experiments follows the paper’s section IV message by message, but departs from it in three places. None changes the operation counts, so none changes the measured cost, and none is hidden: the HMAC is keyed with r1, where the paper keys it with the whole of the data exchanged; AES runs in ECB, where the paper says only shared-key encryption; and the vault update partitions by the 128-bit key length rather than the hash output length, consuming 16 bytes of the digest. Two further departures were removed for this submission because they weakened security rather than cost: challenge indices are now drawn distinct on both sides, and the device

rejects a C2 that repeats an index, exceeds the vault, or selects the same keys as C1 - which the paper requires so that a key recovered for one challenge cannot be reused for the other.

A fourth departure was in the server, and it was also removed. Each protocol used to hold one pending session that any verify request consumed, authentic or not, so an attacker posting garbage between a peer's two requests cancelled that peer's session - the opposite of the paper's claim that no resource is assigned before authentication. A session now carries a random 32-bit identifier, issued by /init and required by /verify, with four slots per protocol, so neither a forged verify nor a flood of /init can disturb a peer that holds one. Lookup is outside every timed phase and the measured costs did not move.

Does it reject what it should?

Measuring only successful authentications would say nothing about whether these are authentication protocols at all, so a second harness sends each of them the requests a faulty or hostile peer would: replay, wrong key, tampered, C2 = C1, C2 repeats, no session, malformed, peer safe. A wrong key is a response computed under key material the device does not hold - for Secure Vault, a vault that never rotated; tampering flips one bit of the authenticating field; a replay resends a body that authenticated once against a fresh challenge; the two C2 cases are the paper's own distinctness requirement; and the last has an attacker post garbage under an identifier it does not hold while a peer has a session open, and passes only if that peer then completes on its own. Every one was refused with the expected status and reason, each protocol's failure counter on the device advanced by exactly the number of requests it refused, and a genuine host-verified cycle succeeded immediately afterwards.

Rejection tests, on the same board and firmware as the measurements. The device's counter is its own tally of refused requests, read over HTTP before and after, and should equal the number of cases sent.

Protocol	Refused as expected	Device counter	Genuine cycle after
Classical AES-128	6 of 6	+6	yes
Rotating password	6 of 6	+6	yes
Secure Vault	8 of 8	+8	yes
ECC (ECDSA P-256)	6 of 6	+6	yes

What the tests cannot show is the property the vault exists for: that recovering one session's key does not predict the next. The paper's Gupta & Kumaraguru's Table 2 states that as $(m - 1) \times 2^n$ brute-force hash operations for m keys of n bits, and four keys of 128 bits per challenge place this implementation at 3×2^{128} - the parameters the claim is made for.

Limitations

The constant-power model charges an accelerated AES block and a software big-integer multiply at the same rate, which understates ECC's true energy share, and it excludes radio energy entirely - in a real deployment the radio, not the cryptography, would dominate all four protocols. Both follow from measuring the algorithm rather than the board. The network path has ten cycles per protocol, enough to corroborate the batch but not to characterise its own dispersion. Two primitives are substituted rather than matched, both disclosed. The ECC baseline is ECDSA sign-plus-verify, chosen because challenge-response maps onto the structure of the other protocols, where the paper says only "ECC public key encryption". The vault update is HMAC-SHA256 [5], where the paper costs

its HMAC row as SHA-512 (their Table 1: 1.5 ms, 149.25 uJ); SHA-256 is the cheaper primitive, so on that term this implementation is if anything favoured.

Conclusion

Secure Vault's design goal - rotating key material so that a captured exchange does not compromise future sessions - is achieved here at symmetric-cryptography cost: 6.0x a two-operation AES challenge-response, 4.0x the paper's own rotating-password baseline, and still 969x cheaper than an asymmetric handshake. That ordering supports the paper's qualitative argument. Its quantitative claim - that the vault is cheaper than rotating a single password - does not hold on hardware that accelerates AES, and most of the gap is already implied by the paper's own per-operation accounting once message size is counted.

References

- [1] M. Gupta and P. Kumaraguru, "Authentication of IoT device and IoT server using secure vaults," in Proc. IEEE TrustCom/BigDataSE, New York, NY, USA, Aug. 2018, pp. 819-824, doi: 10.1109/TrustCom/BigDataSE.2018.00117.
- [2] P. Prasithsangaree and P. Krishnamurthy, "Analysis of energy consumption of RC4 and AES algorithms in wireless LANs," in Proc. IEEE Global Telecommunications Conf. (GLOBECOM), vol. 3, San Francisco, CA, USA, Dec. 2003, pp. 1445-1449, doi: 10.1109/GLOCOM.2003.1258477.
- [3] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS PUB 197, Nov. 2001.
- [4] National Institute of Standards and Technology, "Digital Signature Standard (DSS)," FIPS PUB 186-4, Jul. 2013.
- [5] H. Krawczyk, M. Bellare and R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF RFC 2104, Feb. 1997.
- [6] Espressif Systems, "ESP32-C3 series datasheet," version 2.4. Available: <https://www.espressif.com/en/support/documents/technical-documents>
- [7] Trusted Firmware, "Mbed TLS." Available: <https://github.com/Mbed-TLS/mbedtls>
- [8] E. Risi, "Source code and datasets for this project." Submitted as an archive with this report; also <https://github.com/r15i/secure-vault>

Appendix A - Terminology

Protocols and primitives

Term	Meaning
Classical AES-128	The paper's simplest scheme: one pre-shared key, mutual challenge-response, two AES operations on the device.
Single rotating password	The baseline of Gupta & Kumaraguru's Figure 3: Classical plus a third AES operation carrying the next password.
Secure Vault	The paper's protocol: keys shared by device and server, from which a session key is derived by XOR-ing a challenge-selected subset, and which is rotated after every session.
Vault rotation	The post-session update that changes every key, so a captured exchange does not compromise later ones.
AES-128 (ECB)	Symmetric block cipher, 128-bit key, 16-byte blocks, each encrypted independently; used because the measurement targets the cipher's cost, not a transport mode.
AES operation vs block	The paper bills one operation per encryption or decryption call whatever its length; a block is 16 bytes. The firmware counts both.
HMAC-SHA256	Keyed hash (RFC 2104). Secure Vault uses it over the whole vault to derive the rotation applied after each session.
ECDSA / P-256	Elliptic Curve Digital Signature Algorithm over the NIST P-256 curve; the asymmetric baseline.

Measurement terms

Term	Meaning
Phase	One timed cryptographic step of a protocol (an AES call, the HMAC, a signature). A protocol's per-cycle cost is the sum of its phases.
K	Number of back-to-back repetitions in a batch (1000 for the symmetric protocols, 15 for ECC in this run).
Batch path	The primary measurement: K repetitions of a protocol's phases on the device, reported as the mean per repetition, which drives clock quantisation below the differences being resolved.
Network cycle	One real authentication over HTTP, verified cryptographically by the host. Independent corroboration of the batch figure.
Warm / cold	A batch iteration runs warm, code and tables already in the flash cache; a network cycle runs cold, after the HTTP and WiFi path has evicted them.
Peer simulation	In a batch the device also builds the messages its peer would have sent. That work sits outside the timed phases.

Term	Meaning
Welford’s algorithm	Running mean and variance in constant memory, exact over every sample and needing no sample storage. The median, an order statistic, uses a 256-sample window.
Instrumentation overhead	Measured cost of an empty timed region: the floor below which no phase figure is trustworthy.
Power model	The constant P that turns time into energy: 99.5 mW (paper) or 82.5 mW (device); $E = P t / 1000$, E in uJ, P in mW, t in us.
Mock device	Host-side stand-in running all four protocols with real keys and cryptography but invented timings; marked mock, never reported.

Notation of the exchanges (as in Experiments)

Symbol	Meaning
c, c’	16-byte random challenges issued by the device and by the host
k	the pre-shared AES-128 key of the Classical scheme
p, p’	current and next password of the rotating scheme, 16-byte AES keys
Enc(k, x), Dec(k, x)	AES-128 ECB encryption / decryption of x under key k, per 16-byte block
C1, C2	vault challenges: four indices into the 16-key vault, one set per side
k1, k2	the XOR of the four vault keys that C1 and C2 select
r1, r2	random nonces paired with C1 and C2
t1, t2	random contributions to the session key $t = t1 \text{ XOR } t2$
M3, M4	the paper’s message numbers (M1, M2 are the request and C1 with r1)
, XOR	concatenation; bitwise exclusive-or
HMAC(r1, vault)	HMAC-SHA256 over the whole 256-byte vault, keyed with r1
Sig(X, x)	ECDSA P-256 signature of x under party X’s private key

Appendix B - Supporting figures

B.1 breaks the per-cycle cost down by cryptographic phase; B.2 shows the interface the runs were driven from.

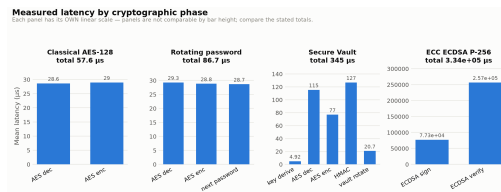


Figure B.1. Latency by cryptographic phase. Each panel has its own linear scale; compare the stated totals, not the bar heights.

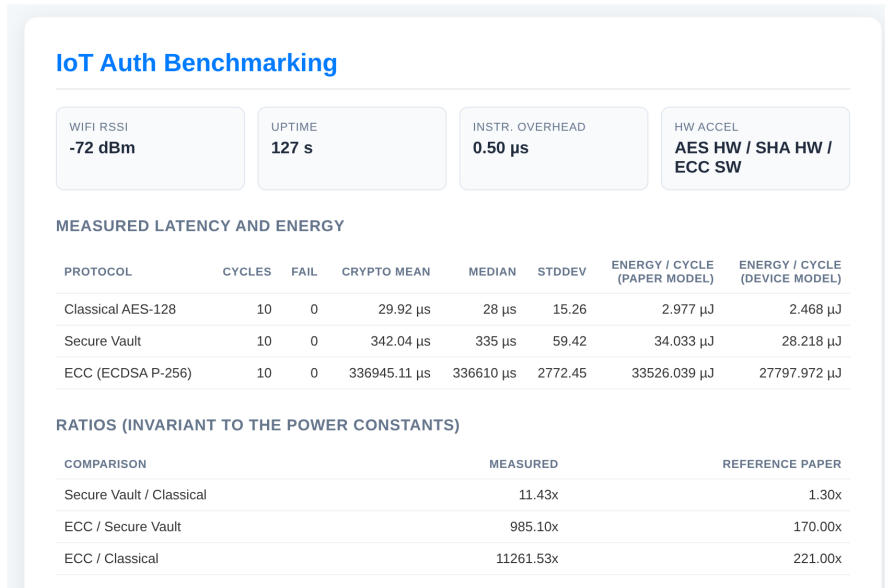


Figure B.2. The dashboard the firmware serves on its root URL, used to drive and watch cycles interactively. It polls `/api/energy` and renders the same per-protocol statistics and ratios the harness records; below the region shown, buttons trigger single cycles and batches on the device.

Appendix C - Reproducing this work

Firmware, harness, analysis and this document accompany this report as an archive, and are also at <https://github.com/r15i/secure-vault> [8]. The datasets were generated by the commands below. Without hardware, `make test-native` checks the measurement arithmetic and `make smoke-test` runs the same pipeline against a mock.

```
git clone https://github.com/r15i/secure-vault.git
cd secure-vault
cp ESP32_Auth_PI0/src/config.example.h ESP32_Auth_PI0/src/config.h # set WIFI_SSID and W
make venv # uv venv + install host deps
make upload PORT=/dev/ttyACMO # flash; monitor prints the IP
make benchmark DEVICE_IP=<ip> # -> results/<run-id>/
make test-security DEVICE_IP=<ip> # -> results/<run-id>/security.json
```